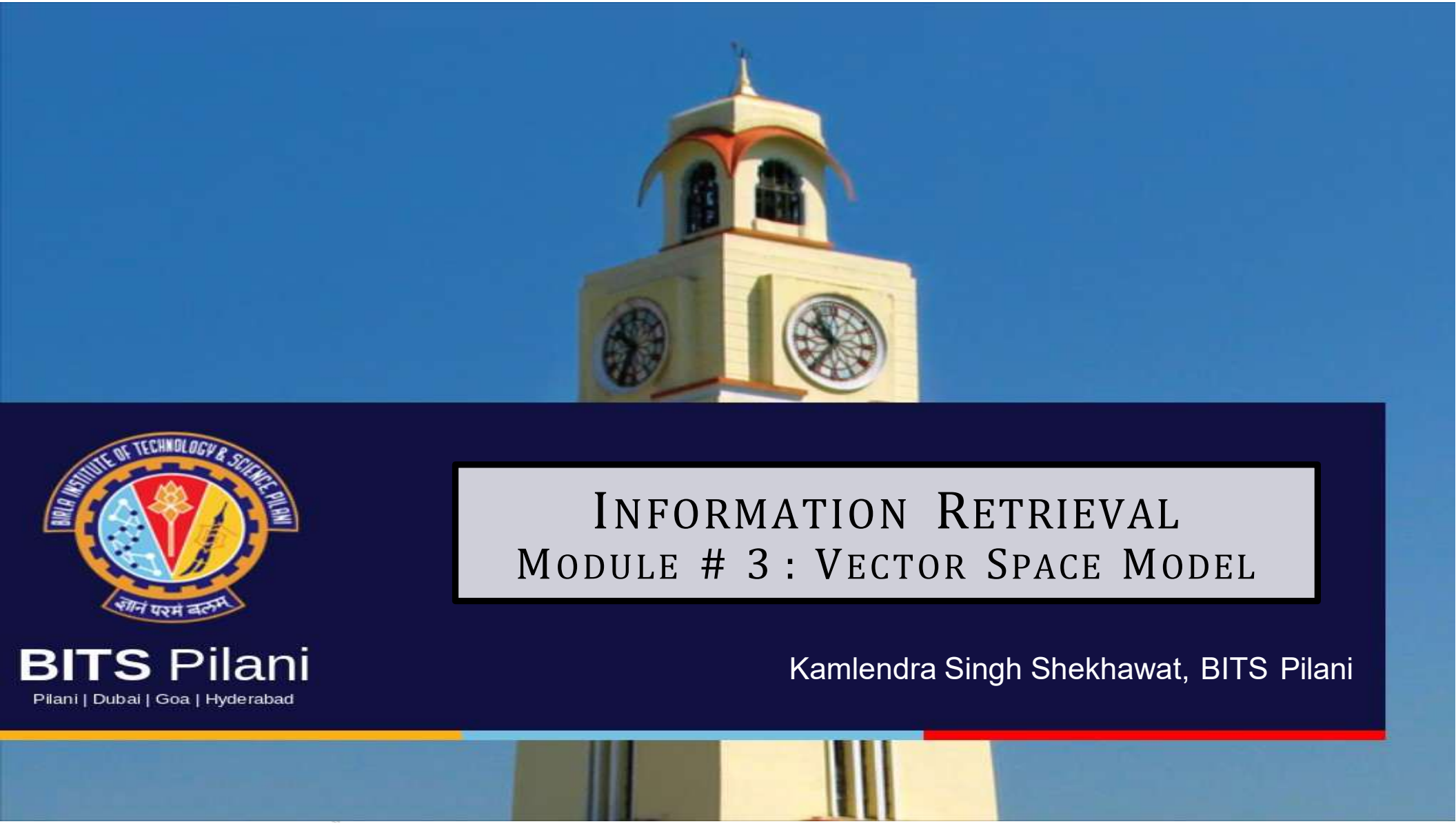

BITS Pilani

Pilani | Dubai | Goa | Hyderabad

INFORMATION RETRIEVAL
MODULE # 3 : VECTOR SPACE MODEL

Kamlendra Singh Shekhawat, BITS Pilani

The instructor is gratefully acknowledging the authors who made their course materials freely available online.

This deck is prepared by Kamlendra Singh Shekhawat.

Work Integrated Learning Programmes

Table of Contents

1. Module 3
2. Why ranked retrieval?
3. Jaccard Coefficient
4. Term frequency
5. IDF weighting
6. Vector Space Model

Module Topics....

- Vector Space Model
  - Scoring, Term Weighting
  - The Vector Space Model for Scoring

Work Integrated Learning Programmes

Table of Contents

1. Module 3
2. Why ranked retrieval?
3. Jaccard Coefficient
4. Term frequency
5. IDF weighting
6. Vector Space Model

Ranked retrieval

- Thus far, our queries have all been Boolean.
  - Documents either match or don't.
- Good for expert users with precise understanding of their needs and of the collection.
  - Also good for applications: Applications can easily consume 1000s of results.
- Not good for the majority of users
  - Most users are not capable of writing Boolean queries or they are, but they think it's too much work.
  - Most users don't want to wade through 1000s of results.
  - This is particularly true of web search.

Boolean Search Query: Feast or famine

- Boolean queries often result in either too few (=0) or too many (1000s) results.
- In Boolean retrieval, it takes a lot of skill to come up with a query that produces a manageable number of hits.
  - AND gives too few.
  - OR gives too many.
- With ranking, large result sets are not an issue.
  - Just show the top 10 results
  - Doesn't overwhelm the user
- Premise: the ranking algorithm works: More relevant results are ranked higher than less relevant results.

Scoring as the Basis of Ranked Retrieval

- We wish to rank documents that are more relevant higher than documents that are less relevant.
- How can we accomplish such a ranking of the documents in the collection with respect to a query?
- Assign a score to each query-document pair, say in [0, 1].
- This score measures how well document and query ”match”.

Query-Document Matching Scores

- How do we compute the score of a query-document pair?
- Let's start with a one-term query.
- If the query term does not occur in the document: score should be 0.
- The more frequent the query term in the document, the higher the score.
- Alternatives for doing this
  - Scoring with the Jaccard coefficient
  - Term frequency weighting
  - Inverse document frequency weighting
  - tf-idf weighting
  - Cosine Similarity
  - Vector space model

Table of Contents

1. Module 3
2. Why ranked retrieval?
3. Jaccard Coefficient
4. Term frequency
5. IDF weighting
6. Vector Space Model

Jaccard Coefficient

- A commonly used measure of overlap of two sets A and B

$$JACCARD(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

- JACCARD(A, A) = 1
- JACCARD(A, B) = 0 if $A \cap B = \emptyset$
- A and B don't have to be the same size.
- Jaccard coefficient always assigns a number between 0 and 1.

Jaccard Coefficient: Example

What is the query-document match score that the Jaccard coefficient computes for:

- Query: ”ides of March”
- Document 1: ”Caesar died in March”
- Document 2: ”the long march”

$$JACCARD(q, d1) = \frac{1}{6}$$

$$JACCARD(q, d2) = ?$$

Issues with Jaccard

- It doesn't consider term frequency (how many occurrences a term has).
  - Rare terms are more informative than frequent terms.
  - Jaccard does not consider this information.
- We need a more sophisticated way of normalizing for the length of a document.
  - Later we'll use Cosine instead of Jaccard for length normalization.

EXERCISE

Compute the Jaccard matching score for the following query-document pairs.

- q: [information on cars] d: ”all you’ve ever wanted to know about cars”
- q: [information on cars] d: ”information on trucks, information on planes, information on trains”
- q: [red cars and red trucks] d: ”cops stop red cars more often”

Table of Contents

1. Module 3
2. Why ranked retrieval?
3. Jaccard Coefficient
4. Term frequency
5. IDF weighting
6. Vector Space Model

Binary Incidence Matrix

Each document is represented as a binary vector $\{0, 1\}^{|V|}$.

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	. . .
ANTHONY	1	1	0	0	1	0	
BRUTUS	1	1	0	1	0	0	
CAESAR	1	1	0	1	1	1	
CALPURNIA	0	1	0	0	0	0	
CLEOPATRA	1	0	0	0	0	0	
MERCY	1	0	1	1	0	1	
WORSER	1	0	1	0	0	0	
. . .							

Term-Document Count Matrices

- Each document is now represented as a count vector.
- It considers the number of occurrences of a term in a document.
- Each document is a count vector $N^{|V|}$: a column below.

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
ANTHONY	157	73	0	2	0	0	
BRUTUS	4	157	0	2	0	0	
CAESAR	232	227	0	0	1	8	
CALPURNIA	0	10	0	0	0	0	
CLEOPATRA	57	0	0	0	0	0	
MERCY	2	0	3	8	5	0	
WORSER	2	0	1	1	1	0	

Bag of Words Model

- We do not consider the order of words in a document.
- John is quicker than Mary and Mary is quicker than John are represented the same way.
- This is called a bag of words model.
- In a sense, this is a step back: The positional index was able to distinguish these two documents.
- For now: bag of words model

Term Frequency *tf*

- The term frequency $tf_{t,d}$ of term *t* in document *d* is defined as the number of times that *t* occurs in *d*.
- How to use *tf* when computing query-document match scores?
- Raw term frequency is not what we want because:
  - A document with *tf* = 10 occurrences of the term is more relevant than a document with *tf* = 1 occurrence of the term. (Frequency = count in IR)
  - But not 10 times more relevant.
- Relevance does not increase proportionally with term frequency.

Log-Frequency Weighing

- The log frequency weight $w_{t,d}$ of term t in document d

$$w_{t,d} = \begin{cases} 1 + \log_{10} \mathrm{tf}_{t,d} & \text{if } \mathrm{tf}_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$$

- Score for the document-query pair = sum over terms t in both 1 and d.

$$score = \sum_{t \in q \cap d} 1 + \log_{10} \mathrm{tf}_{t,d}$$

- The score is 0 if none of the query terms matches in the document.

EXERCISE

Compute the *tf* matching (Log-Frequency Weighing) score for the following query-document pairs.

- q: [information on cars] d: ”all you’ve ever wanted to know about cars”
- q: [information on cars] d: ”information on trucks, information on planes, information on trains”
- q: [red cars and red trucks] d: ”cops stop red cars more often”

Table of Contents

1. Module 3
2. Why ranked retrieval?
3. Jaccard Coefficient
4. Term frequency
5. IDF weighting
6. Vector Space Model

Document Frequency

- Rare terms are more informative than frequent terms.
- Consider a term in the query that is rare in the collection (e.g., ARACHNOCENTRIC).
- A document containing this term is very likely to be relevant.
- We want high weights for rare terms like ARACHNOCENTRIC.

Desired Weight for Frequent Terms

- Frequent terms are less informative than rare terms.
- Consider a term in the query that is frequent in the collection (e.g., GOOD, INCREASE, LINE).
- A document containing this term is more likely to be relevant than a document that doesn't.
- → For frequent terms like GOOD, INCREASE and LINE, we want positive weights but lower weights than for rare terms.
- We will use document frequency to factor this into computing the matching score.
- The document frequency is the number of documents in the collection that the term occurs in.

IDF WEIGHT

- df_t is the document frequency of t, the number of documents that t occurs in.
  - df_t is an inverse measure of the informativeness of term t.
  - $df_t \leq N$
- We define the idf weight of term t as follows:

$$idf_t = \log_{10} \frac{N}{df_t}$$

(*N* is the number of documents in the collection.)

- idf_t is a measure of the informativeness of the term.
- $[\log \frac{N}{df_t}]$ instead of $[\frac{N}{df_t}]$ to "dampen" the effect of idf.

EXAMPLES FOR IDF

Compute idf_t using the formula: $idf_t = \log_{10} \frac{N}{df_t}$

term	df_t	idf_t
calpurnia	1	6
animal	100	4
sunday	1,000	3
fly	10,000	2
under	100,000	1
the	1,000,000	0

Collection Frequency vs. Document Frequency

- Collection frequency of *t*: number of occurrences of *t* in the collection.
- Document frequency of *t*: number of documents *t* occurs in.
- Which word is a better search term (and should get a higher weight)?

word	collection frequency	document frequency
INSURANCE	10440	3997
TRY	10422	8760

This example suggests that df (and idf) is better for weighting than cf (and "icf").

TF-IDF WEIGHTING

- The tf-idf weight of a term is the product of its tf weight and its idf weight.

$$\text{tf-idf}_t = (1 + \log_{10} \text{tf}_{t,d}) \times \log_{10} \frac{N}{\text{df}_t}$$

$$score(q, d) = \sum_{t \in q \cap d} \text{tf.idf}_{t,d})$$

- Best known weighting scheme in information retrieval. Alternative names: tf.idf, tf x idf.
- The tf-idf weight
  - Increases with the number of occurrences within a document. (term frequency)
  - Increases with the rarity of the term in the collection. (inverse document frequency)

Term, Collection, Document Frequency

Find the

- Relationship between df and cf?
- Relationship between tf and cf?
- Relationship between tf and df?

Quantity	Symbol	Definition
term frequency	$tf_{t,d}$	number of occurrences of *t* in *d*
document frequency	df_t	number of documents in the collection that *t* occurs in
collection frequency	cf_t	total number of occurrences of *t* in the collection

Table of Contents

1. Module 3
2. Why ranked retrieval?
3. Jaccard Coefficient
4. Term frequency
5. IDF weighting
6. Vector Space Model

Binary → Count → Weight Matrix

Each document is now represented as a real-valued vector of tf-idf weights $R^{|V|}$.

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
ANTHONY	5.25	3.18	0.0	1.0	0.0	0.0
BRUTUS	1.21	6.10	0.0	1.51	0.0	0.0
CAESAR	8.59	2.54	0.0	0.0	0.25	0.88
CALPURNIA	0.0	1.54	0.0	0.0	0.0	0.0
CLEOPATRA	2.85	0.0	0.0	0.0	0.0	0.0
MERCY	1.51	0.0	1.90	4.15	5.25	0.0
WORSER	1.37	0.0	0.11	0.12	0.25	0.0

Documents as vectors

- Each document is now represented as a real-valued vector of tf-idf weights $R^{|V|}$.
- So we have a $|V|$-dimensional real-valued vector space.
- Terms are axes of the space.
- Documents are points or vectors in this space.
- Very high-dimensional: tens of millions of dimensions when you apply this to web search engines
- Each vector is very sparse - most entries are zero.

Queries as Vectors

- Key idea 1: do the same for queries: represent them as vectors in the high-dimensional space
- Key idea 2: Rank documents according to their proximity to the query
  - proximity = similarity of vectors
  - proximity $\approx$ inverse of distance
- Rank more relevant documents higher than less relevant documents

Formalize Vector Space Similarity

- First cut: distance between two points
  - (= distance between the end points of the two vectors)
- Euclidean distance?
  - Euclidean distance is a bad idea . . .
  - . . . because Euclidean distance is large for vectors of different lengths.

Use Angle instead of Distance

- Key idea: Rank documents according to angle with query.
- Thought experiment: take a document d and append it to itself. Call this document d^r.
- d^r is twice as long as d.
- "Semantically" d and d^r have the same content.
- The angle between the two documents is 0, corresponding to maximal similarity . . .
- . . . even though the Euclidean distance between the two documents can be quite large.

Cosine Similarity between Query and Document

- q_i is the tf-idf weight of term i in the query.
- d_i is the tf-idf weight of term i in the document.
- $|q|$ and $|d|$ are the lengths of q and d.
- Cosine similarity of q and d, equivalently, the cosine of the angle between q and d

$$\text{sim}(q, d) = \frac{\sum_{i=1}^{V} q_i d_i}{\sqrt{\sum_{i=1}^{V} q_i^2}\sqrt{\sum_{i=1}^{V} d_i^2}} = \cos\vartheta$$

Cosine for Normalized Vectors

For normalized vectors, the cosine is equivalent to the dot product or scalar product.

$$\text{sim}(q, d) = \vec{q} \cdot \vec{d} = \sum_{i=1}^{V} q_i \cdot d_i$$

(if q and d are length-normalized).

Cosine: Example

How similar are these novels?

SaS: Sense and Sensibility

PaP: Pride and Prejudice

WH: Wuthering Heights

term frequencies (counts)

term	SaS	PaP	WH
AFFECTION	115	58	20
JEALOUS	10	7	11
GOSSIP	2	0	6
WUTHERING	0	0	38

Cosine: Example

term	term frequencies (counts)			log frequency weighting		
	SaS	PaP	WH	SaS	PaP	WH
AFFECTION	115	58	20	3.06	2.76	2.30
JEALOUS	10	7	11	2.0	1.85	2.04
GOSSIP	2	0	6	1.30	0	1.78
WUTHERING	0	0	38	0	0	2.58

(To simplify this example, we don't do idf weighting.)

Cosine: Example

term	log frequency weighting SaS	PaP	WH	cosine normalization SaS	PaP	WH
AFFECTION	3.06	2.76	2.30	0.789	0.832	0.524
JEALOUS	2.0	1.85	2.04	0.515	0.555	0.465
GOSSIP	1.30	0	1.78	0.335	0.0	0.405
WUTHERING	0	0	2.58	0.0	0.0	0.588

- $\cos(\text{SaS}, \text{PaP}) \approx 0.789 \cdot 0.832 + 0.515 \cdot 0.555 + 0.335 \cdot 0.0 + 0.0 \cdot 0.0 \approx 0.94$
- $\cos(\text{SaS}, \text{WH}) \approx 0.79$
- $\cos(\text{PaP}, \text{WH}) \approx 0.69$

Why do we have $\cos(\text{SaS}, \text{PaP}) > \cos(\text{SAS}, \text{WH})$?

Variants of tf-idf Weighting

Term frequency		Document frequency		Normalization	
n (natural)	$\mathrm{tf}_{t,d}$	n (no)	1	n (none)	1
l (logarithm)	$1+\log(\mathrm{tf}_{t,d})$	t (idf)	$\log \frac{N}{\mathrm{df}_t}$	c (cosine)	$\frac{1}{\sqrt{w_1^2+w_2^2+\ldots+w_M^2}}$
a (augmented)	$0.5+\frac{0.5\times \mathrm{tf}_{t,d}}{\max_t(\mathrm{tf}_{t,d})}$	p (prob idf)	$\max\{0, \log \frac{N-\mathrm{df}_t}{\mathrm{df}_t}\}$	u (pivoted unique)	$1/u$
b (boolean)	$\begin{cases}1 & \text{if } \mathrm{tf}_{t,d}>0\\ 0 & \text{otherwise}\end{cases}$			b (byte size)	$1/CharLength^{\alpha}$, $\alpha<1$
L (log ave)	$\frac{1+\log(\mathrm{tf}_{t,d})}{1+\log(\mathrm{ave}_{t\in d}(\mathrm{tf}_{t,d}))}$				

TF-IDF EXAMPLE

- We often use different weightings for queries and documents.
- Notation: ddd.qqq
- Example: lnc.ltn
  - document: logarithmic tf, no df weighting, cosine normalization
  - query: logarithmic tf, idf, no normalization

Similarity Scores

Dot product $\mathrm{Sim}(Q, D) = \sum_i a_i b_i$

Cosine $\mathrm{Sim}(Q, D) = \dfrac{\sum_i a_i b_i}{\sqrt{\sum a_i^2}\sqrt{\sum_i b_i^2}}$

Dice $\mathrm{Sim}(Q, D) = \dfrac{2\sum_i a_i b_i}{\sum_i a_i^2 + \sum_i b_i^2}$

Jaccard $\mathrm{Sim}(Q, D) = \dfrac{\sum_i a_i b_i}{\sum_i a_i^2 + \sum_i b_i^2 - \sum_i a_i b_i}$

Summary: Ranked Retrieval in Vector Space Model

- Represent the query as a weighted tf-idf vector
- Represent each document as a weighted tf-idf vector
- Compute the cosine similarity between the query vector and each document vector
- Rank documents with respect to the query
- Return the top K (e.g., K = 10) to the user

Comments on Vector Space Models

- Simple, mathematically based approach.
- Considers both local (tf) and global (idf) word occurrence frequencies.
- Provides partial matching and ranked results.
- Tends to work quite well in practice despite obvious weaknesses.
- Allows efficient implementation for large document collections.

Problems with Vector Space Model

- Missing semantic information (e.g. word sense).
- Missing syntactic information (e.g. phrase structure, word order, proximity information).
- Assumption of term independence (e.g. ignores synonomy).
- Lacks the control of a Boolean model (e.g., requiring a term to appear in a document).
  - Given a two-term query ”A B”, may prefer a document containing A frequently but not B, over a document that contains both A and B, but both less frequently.

References

1. Chapter 1 to 5
   C. D. Manning, P. Raghavan and H. Schutze. Introduction to Information Retrieval. 2008.
   http://nlp.stanford.edu/IR-book/

Thank You!